

Resolución de Sudokus con Método de Enteros Binarios

Marco Fernández

Metodos Numericos

UCA

Febrero 2025

Resumen

El objetivo principal de este trabajo es estudiar y aplicar técnicas matemáticas de optimización para resolver el conocido juego de lógica Sudoku, originario de Suiza y ampliamente popularizado a nivel mundial. Para abordar este problema, hemos elegido el enfoque de programación de enteros binarios, un método que permite modelar el tablero de Sudoku en tres dimensiones. Este enfoque aprovecha la representación binaria para verificar el cumplimiento de las restricciones inherentes al juego, como la unicidad de los números en filas, columnas y subcuadrados.

Inicialmente, se analiza un Sudoku predefinido como caso de prueba, donde se validan las restricciones y la solución obtenida mediante la herramienta de optimización matemática utilizada. Posteriormente, adaptaremos este enfoque para generar y resolver Sudokus generados al azar, lo que implica la necesidad de diseñar algoritmos que creen tableros iniciales válidos respetando las reglas del juego. Este estudio no solo pone a prueba las capacidades del modelo propuesto, sino que también resalta los desafíos asociados con la generación de tableros iniciales que sean tanto válidos como solubles.

En este informe, se presenta una discusión detallada sobre la implementación computacional del modelo, las estrategias adoptadas para garantizar la validez de las soluciones, y los resultados obtenidos al aplicar el enfoque en diferentes instancias del problema.

Índice

1. Introducción	3
2. Metodología	4
2.1. Enfoque de Programación de Enteros Binarios	4
2.2. Modelado y Validación de Restricciones	4
2.3. Implementación del Código	5
2.4. Generación de Sudokus mediante Backtracking	6
2.5. Optimización y Estrategias de Resolución	6
3. Resultados	8
4. Conclusión	9

1 Introducción

El Sudoku es un popular juego de lógica que ha capturado la atención de millones de personas alrededor del mundo. Su historia remonta al siglo XVIII, con la creación de los "cuadrados mágicos", matrices numéricas que respetaban ciertas propiedades matemáticas. Sin embargo, el formato moderno del Sudoku fue desarrollado en 1979 por Howard Garns bajo el nombre Number Place y más tarde popularizado en Japón en la década de 1980, donde recibió el nombre que hoy conocemos: Sudoku, abreviatura de una frase japonesa que significa "los números deben estar solos".

El Sudoku estándar se juega en un tablero de 9×9 , dividido en nueve subcuadros de 3×3 . El objetivo es rellenar las celdas vacías con dígitos del 1 al 9, de manera que cada fila, columna y subcuadro contenga exactamente una vez cada número del conjunto. Aunque las reglas son simples, la resolución del Sudoku puede variar desde ejercicios triviales hasta desafíos computacionales significativos, dependiendo de la cantidad y distribución de los números iniciales proporcionados, denominados "pistas".

Resolver un Sudoku de forma manual implica estrategias lógicas como la eliminación de posibilidades en celdas individuales, análisis de patrones y prueba y error. No obstante, desde el punto de vista computacional, el Sudoku es un problema interesante porque, cuando el tamaño del tablero es variable, pertenece a la clase de problemas NP-completos.

En este trabajo, utilizamos técnicas de programación de enteros binarios para modelar y resolver el Sudoku. Este enfoque representa cada celda del tablero como una variable binaria tridimensional, donde la tercera dimensión corresponde a los posibles valores (1 a 9). Las reglas del juego se traducen en un conjunto de restricciones lineales, tales como que la suma de las variables binarias en cada fila, columna y subcuadro sea igual a uno. La programación de enteros binarios ofrece una solución sistemática y garantizada, permitiendo abordar no solo la resolución de Sudokus preexistentes, sino también la generación automática de tableros iniciales válidos y solubles.[5]

El objetivo principal del modelo es evaluar la efectividad del método de programación de enteros binarios para resolver Sudokus con diferentes niveles de vaciado. Se busca generar Sudokus válidos y solubles, resolverlos con niveles de dificultad variables (entre 30 % y 98 % de celdas vacías), y medir la eficiencia del método en términos de tiempo de resolución y tasa de éxito. Además, se identifican los límites del modelo, especialmente en casos extremos como Sudokus con un 98 % de celdas vacías.

El método propuesto no solo tiene aplicaciones recreativas, sino que también sirve como un

ejemplo para ilustrar el uso de herramientas matemáticas avanzadas en problemas combinatorios. Además, destaca cómo enfoques algorítmicos pueden ser adaptados para resolver problemas de decisión y optimización en áreas tan diversas como la inteligencia artificial, la planificación logística y la investigación operativa.

En las siguientes secciones, se presenta una descripción detallada del modelo matemático, su implementación computacional y los resultados obtenidos al aplicar este enfoque a diferentes instancias del problema del Sudoku. Este análisis no solo refuerza la utilidad de la programación de enteros binarios en la resolución de problemas complejos, sino que también aporta una comprensión más profunda de las estructuras matemáticas subyacentes a este popular juego.

2 Metodología

2.1 Enfoque de Programación de Enteros Binarios

La programación de enteros binarios es una técnica matemática utilizada para modelar problemas en los cuales las variables de decisión solo pueden tomar los valores 0 o 1. Este enfoque es ideal para resolver el Sudoku, ya que permite representar la solución del tablero mediante un arreglo tridimensional binario $x(i, j, k)$, donde:

- i y j representan las coordenadas de la celda en el tablero de 9×9 ,
- k indica el valor que puede tomar la celda, variando entre 1 y 9.

La variable binaria $x(i, j, k)$ toma el valor 1 si el número k se encuentra en la celda (i, j) , y 0 en caso contrario. Esto transforma el problema de resolver el Sudoku en uno de encontrar una combinación de valores binarios que cumpla las restricciones del juego.

2.2 Modelado y Validación de Restricciones

Para garantizar que cualquier solución propuesta cumpla con las reglas del Sudoku, se establecen las siguientes restricciones como ecuaciones lineales:[1]

1. **Una celda contiene un único valor:** Cada celda (i, j) del tablero debe contener exactamente un número del 1 al 9:

$$\sum_{k=1}^9 x(i, j, k) = 1, \quad \forall i, j.$$

2. **Valores únicos en cada fila:** Cada número del 1 al 9 debe aparecer exactamente una vez en cada fila i :

$$\sum_{j=1}^9 x(i, j, k) = 1, \quad \forall i, k.$$

3. **Valores únicos en cada columna:** Cada número del 1 al 9 debe aparecer exactamente una vez en cada columna j :

$$\sum_{i=1}^9 x(i, j, k) = 1, \quad \forall j, k.$$

4. **Restricciones en los subcuadros:** Cada subcuadro de 3×3 debe contener exactamente un número del 1 al 9. Esto se logra sumando las variables $x(i, j, k)$ correspondientes a las celdas dentro del subcuadro:

$$\sum_{i \in \text{subcuadro}, j \in \text{subcuadro}} x(i, j, k) = 1, \quad \forall k.$$

Estas restricciones aseguran que cualquier solución obtenida sea válida según las reglas del Sudoku.

2.3 Implementación del Código

El proceso de resolución se implementa utilizando herramientas de optimización en MATLAB. Se define el problema como un objeto de optimización, y se construyen las restricciones previamente mencionadas. La función objetivo utilizada no tiene un propósito práctico en este caso, ya que el objetivo principal es satisfacer todas las restricciones.

El modelo sigue estos pasos generales:

- Paso 1 - Definición de las variables:** Se crea una variable tridimensional binaria $x(i, j, k)$, donde i, j corresponden a las coordenadas del tablero y k representa los valores posibles.
- Paso 2 - Modelado de las restricciones:** Se agregan las restricciones necesarias para garantizar que cada celda, fila, columna y subcuadro cumpla las reglas del Sudoku.
- Paso 3 - Resolución del problema:** Se utiliza un solucionador de problemas de optimización para encontrar los valores binarios que satisfacen las restricciones.
- Paso 4 - Reconstrucción de la solución:** El arreglo tridimensional resultante se procesa para generar un tablero bidimensional que representa la solución final del Sudoku.

Paso 5 - Visualización: Se utiliza una función personalizada para graficar el tablero de Sudoku resuelto.

El código permite resolver tanto Sudokus preexistentes como generar nuevos tableros válidos al agregar números iniciales respetando las restricciones. Este enfoque es una demostración del poder de las técnicas matemáticas de optimización para resolver problemas combinatorios complejos.

2.4 Generación de Sudokus mediante Backtracking

El *método de backtracking* es una técnica algorítmica ampliamente utilizada para resolver problemas que implican la exploración de múltiples combinaciones posibles de una solución. En el contexto de los Sudokus, este método se emplea para generar tableros completos y válidos siguiendo las reglas fundamentales del juego.

El proceso de generación comienza con un tablero vacío. A partir de ahí, el algoritmo inserta un número en una celda vacía y verifica si esta asignación cumple con las reglas del Sudoku. Si la asignación es válida, el algoritmo avanza a la siguiente celda vacía y repite el proceso. Sin embargo, si en algún punto no es posible encontrar un número válido, el algoritmo retrocede (*backtracking*) a la celda anterior, deshace la asignación previa y prueba con otro número. Este enfoque asegura que cada solución parcial explorada siempre sea consistente con las reglas del juego.

Este proceso continúa hasta que el tablero se complete en su totalidad, garantizando así que el Sudoku generado sea resoluble. El uso del método de *backtracking* asegura que todos los tableros generados sean correctos, únicos y cumplan estrictamente las reglas del Sudoku, ya que en cada paso se verifica la validez antes de proceder. Una vez que el tablero está lleno, se eliminan celdas de manera controlada hasta alcanzar el porcentaje deseado de casillas vacías, obteniendo así un tablero inicial válido. Finalmente, la representación del Sudoku se transforma a un formato tridimensional de dimensiones $9 \times 9 \times 9$, lo que permite su resolución mediante el método de enteros binarios. Este enfoque garantiza una generación confiable y adaptable de Sudokus con distintos niveles de complejidad.

2.5 Optimización y Estrategias de Resolución

El modelo de Sudoku se formula definiendo 729 variables binarias $x(i, j, k)$ que representan la posible asignación del dígito k a la celda (i, j) del tablero. Aunque el objetivo principal es encontrar una solución factible que satisfaga todas las restricciones (unicidad en celdas,

filas, columnas y subcuadros), se define una función objetivo arbitraria:

$$\text{mín} \sum_{i=1}^9 \sum_{j=1}^9 \sum_{k=1}^9 k \cdot x(i, j, k).$$

Esta función cumple dos propósitos esenciales:

- a) **Romper simetrías:** El Sudoku presenta numerosas soluciones equivalentes debido a permutaciones de dígitos. La función objetivo orienta la búsqueda hacia una solución preferente, evitando que el solver dedique recursos a explorar ramas redundantes.
- b) **Guía en el proceso :** Aunque cualquier solución válida satisface las restricciones del problema, el objetivo definido proporciona un criterio cuantitativo que permite al algoritmo evaluar y priorizar las posibles asignaciones durante el proceso de optimización.

El proceso de optimización se lleva a cabo mediante métodos especializados, entre los que destacan:

- **Relajación Lineal:** Inicialmente, se relaja la condición de integridad permitiendo que $x(i, j, k) \in [0, 1]$. Esta relajación se resuelve mediante técnicas de programación lineal, obteniéndose un límite (en problemas de minimización, un límite inferior) que orienta la búsqueda de la solución entera.
- **Branch-and-Bound:** Si la solución de la relajación presenta variables fraccionarias, se aplica la técnica de *branch-and-bound*. Este método consiste en:[3]
 - **Ramificación:** Se selecciona una variable no entera y se crean dos subproblemas, fijando dicha variable a 0 y a 1, respectivamente.
 - **Acotamiento:** Se resuelven las relajaciones de los subproblemas para obtener cotas que permitan descartar ramas del árbol de decisión que no pueden conducir a una solución mejor.
- **Branch-and-Cut:** Además del branch-and-bound, se incorporan cortes o *cutting planes* que eliminan regiones del espacio factible con soluciones fraccionarias sin excluir ninguna solución entera válida, acelerando así la convergencia. [4]
- **Heurísticas y Preprocesamiento:** Los solvers modernos, como *intlinprog* en MATLAB, implementan estrategias heurísticas que identifican rápidamente soluciones factibles y técnicas de preprocesamiento que reducen el tamaño del problema mediante la eliminación de redundancias y la detección de simetrías.

El uso de estos métodos tiene como finalidad maximizar la eficiencia computacional y la robustez del algoritmo, garantizando que la solución obtenida no solo cumpla todas las restricciones del Sudoku, sino que también se encuentre en un tiempo razonable, incluso en

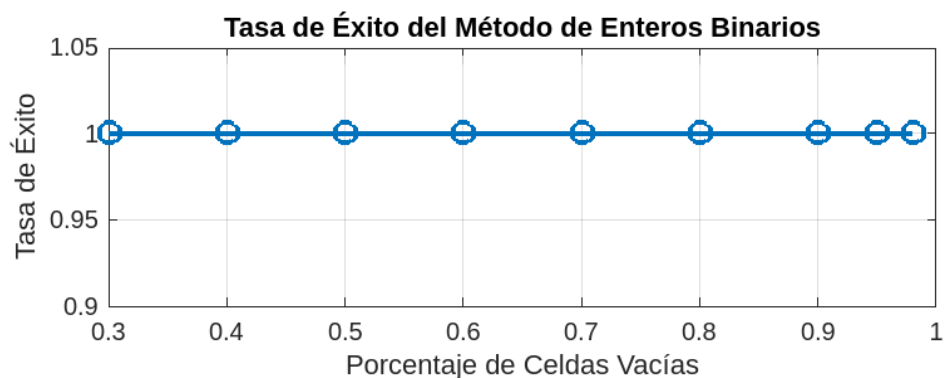


Figura 1: Comparacion de Tasa de Exito

escenarios con un elevado porcentaje de celdas vacías. La estrategia de definir una función objetivo, aun siendo en principio arbitraria, resulta fundamental para orientar la búsqueda en el vasto espacio combinatorio inherente a este problema.

3 Resultados

Los resultados obtenidos evidencian que el método de programación entera binaria es efectivo para resolver todos los sudokus que admiten una solución. La capacidad computacional utilizada en la resolución supera ampliamente la complejidad inherente del problema, lo que garantiza que, incluso aplicando un enfoque de fuerza bruta basado únicamente en las restricciones mínimas del juego, es posible encontrar la solución en todos los casos, independientemente de su nivel de dificultad.

No obstante, los experimentos permiten identificar un punto crítico en el que el modelo alcanza su máxima dificultad. Específicamente, cuando aproximadamente el 70 % de las celdas del sudoku han sido vaciadas, el tiempo promedio de resolución alcanza su valor máximo. Previo a este umbral, el tiempo de cómputo se mantiene relativamente constante, mientras que, una vez superado, comienza a disminuir.

Este comportamiento sugiere que el sudoku más difícil de resolver no es aquel completamente vacío ni aquel completamente lleno, sino aquel que presenta un equilibrio entre restricciones y posibilidades. En otras palabras, en un estado intermedio donde existen suficientes restricciones para evitar soluciones triviales, pero también suficientes espacios vacíos como para generar un alto número de combinaciones posibles, la resolución se vuelve más compleja desde un punto de vista

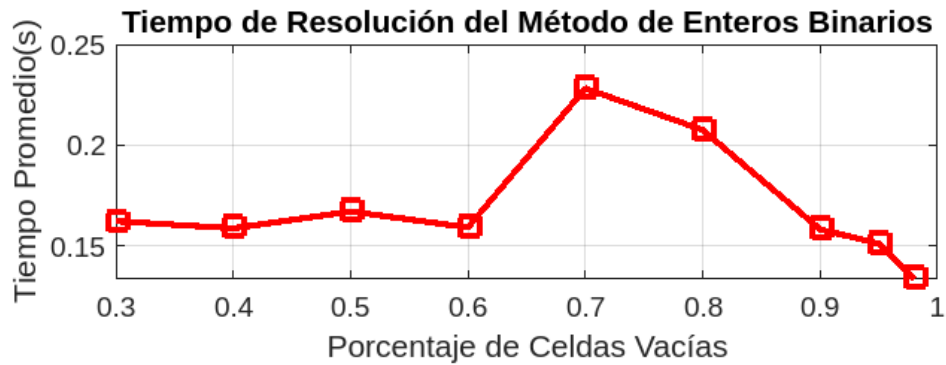


Figura 2: Comparacion de Tiempo de ejecucion

4 Conclusión

El método de programación de enteros binarios se consolida como una herramienta poderosa y sistemática para resolver Sudokus, transformando un problema de lógica combinatoria en un modelo matemático riguroso. Al representar cada celda como una variable tridimensional binaria y traducir las reglas del juego en restricciones lineales, este enfoque garantiza soluciones válidas y únicas, incluso en Sudokus con hasta un 98% de celdas vacías. La optimización matemática, respaldada por técnicas como branch-and-bound y branch-and-cut, permite navegar eficientemente el vasto espacio de soluciones, reduciendo tiempos de cómputo y evitando redundancias.

Los resultados demuestran que la dificultad máxima no radica en tableros completamente llenos o vacíos, sino en un equilibrio crítico (alrededor del 70% de celdas vacías), donde las restricciones y posibilidades se entrelazan de forma compleja. Este método no solo resuelve Sudokus de manera robusta, sino que también ilustra cómo problemas aparentemente lúdicos pueden abordarse con herramientas avanzadas de investigación operativa, abriendo puertas a aplicaciones en logística, inteligencia artificial y más. La programación de enteros binarios emerge así como un puente entre la teoría matemática y los desafíos prácticos del mundo real.

Referencias

- [1] Bartlett, A. C., Chartier, T. P., Langville, A. N., and Rankin, T. D. (2007). *An integer programming model for the Sudoku problem*. Journal of Online Mathematics and its Applications.
- [2] Wolsey, L. A. (1998). *Integer programming*. Wiley.

- [3] Nemhauser, G. L., and Wolsey, L. A.(1988). *Integer and combinatorial optimization*. Wiley.
- [4] Padberg, M (1991). *A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems*. SIAM Review
- [5] MathWorks. (2024). *Sudoku puzzles, problem-based*. MATLAB Documentation. Recuperado el 18 de julio de 2024, de <https://la.mathworks.com/help/optim/ug/sudoku-puzzles-problem-based.html>